Masked Grouped GEMM

Expert GEMMs whose token counts never leave the GPU

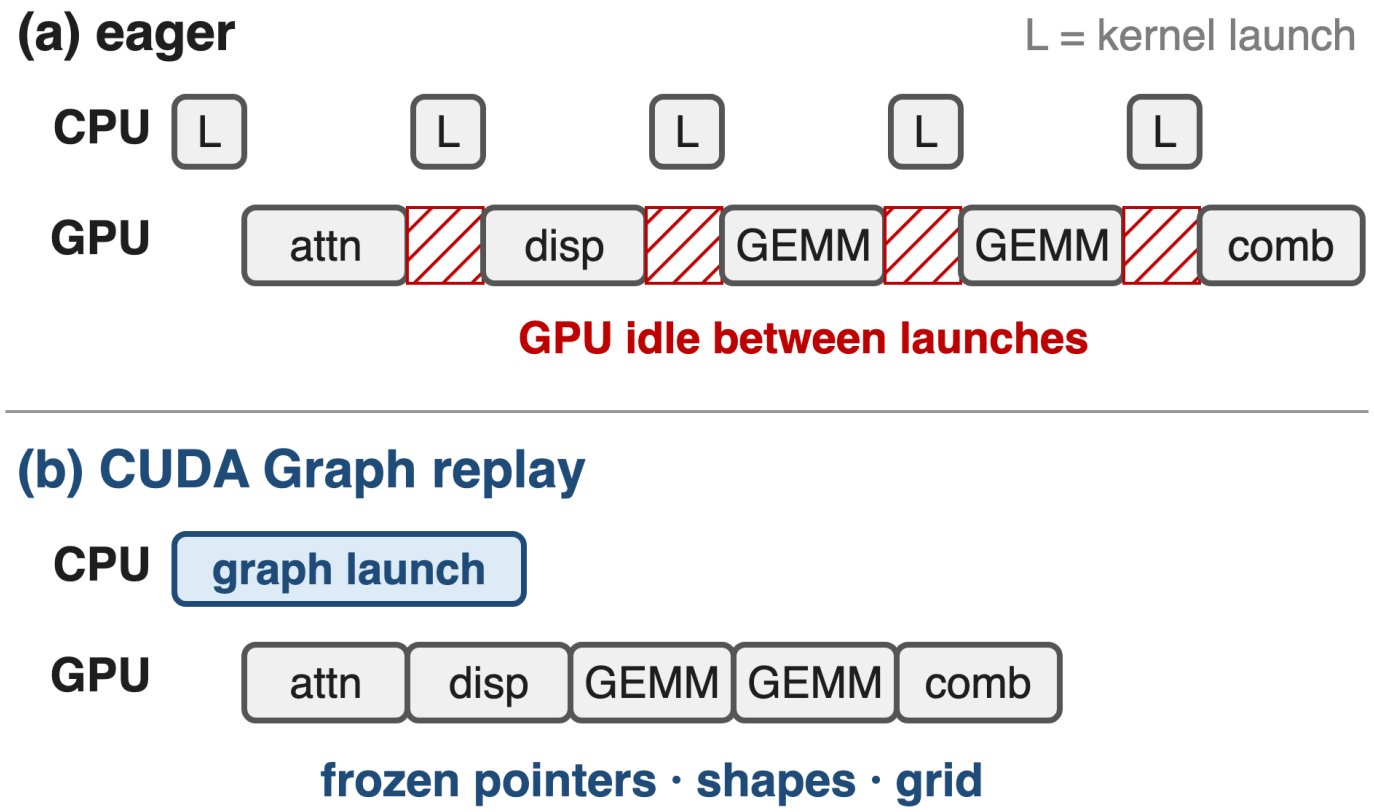
DeepSeek-AI · DeepGEMM masked layout + DeepEP low-latency kernels

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

GEMM & MoE Kernels · github.com/deepseek-ai/DeepGEMM

Decode runs tiny MoE layers as CUDA Graphs

- DeepSeek-V3 decode: EP320, ≤ **256** tokens per expert
- Memory-bound kernels; each launch costs CPU µs
- A graph replays a whole step with one launch



Host-shaped GEMMs cost 58 syncs per step

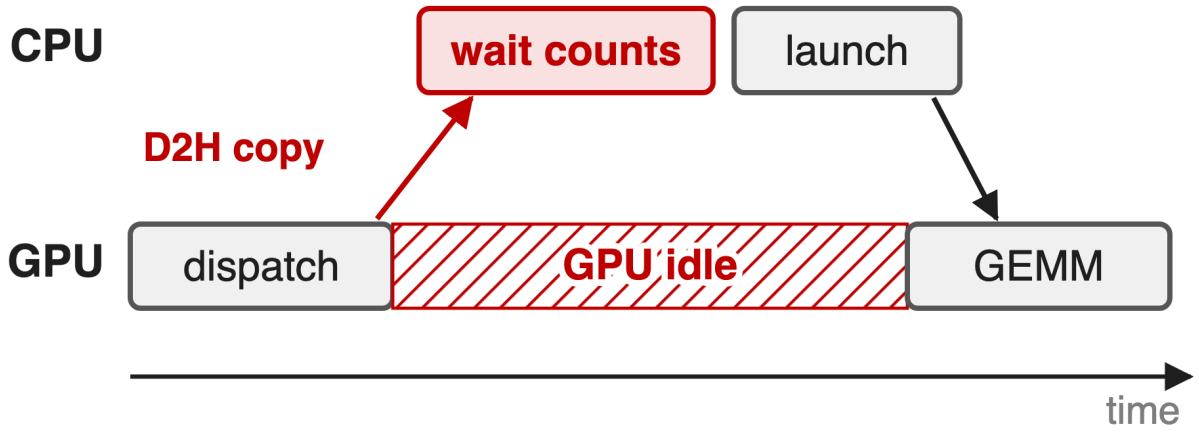
58

blocking reads of expert token counts per DeepSeek-V3 decode step, one per MoE layer, if the GEMM is shaped on the host

Derived: 58 MoE layers (DeepSeek-V3 §4.2; MCore report §4.2.1) × one count read per layer (MCore §4.3.7 p.43; DeepEP README: normal dispatch makes the CPU wait)

Host shapes stall; worst-case shapes waste

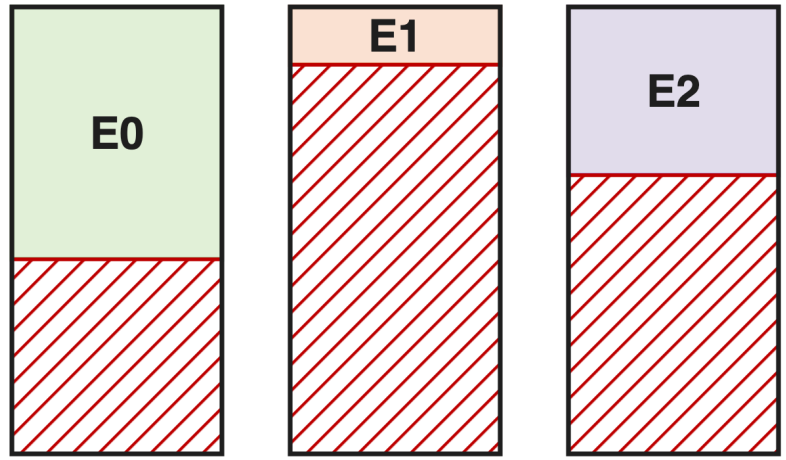
(a) GEMM shaped on the host



once per MoE layer

shape depends on GPU data: no capture

(b) Batched GEMM at worst case



every capacity row computed

graph-safe but wasteful

Either the CPU waits every layer, or every slot row is computed

Insight: graphs freeze arguments, not memory

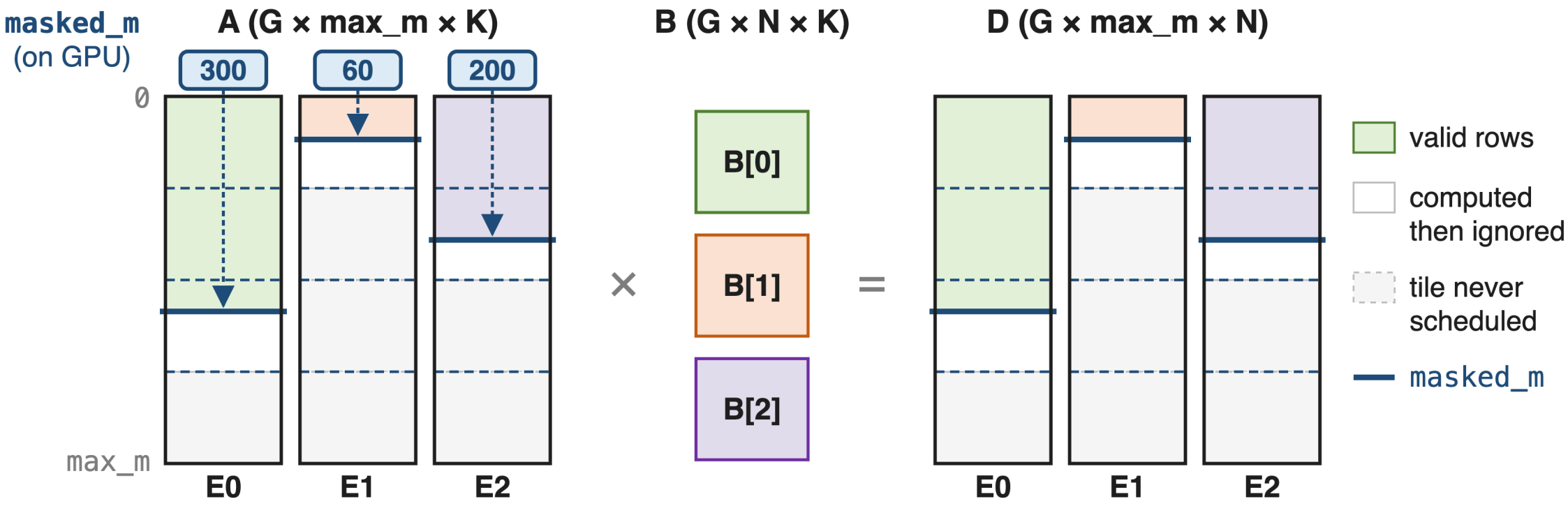
- Captured: pointers, shapes, grid size
- Not captured: what kernels read from GPU memory

Insight: graphs freeze arguments, not memory

- Captured: pointers, shapes, grid size
- Not captured: what kernels read from GPU memory

Keep shapes at a fixed capacity, keep per-expert counts on the GPU, let the kernel skip invalid rows

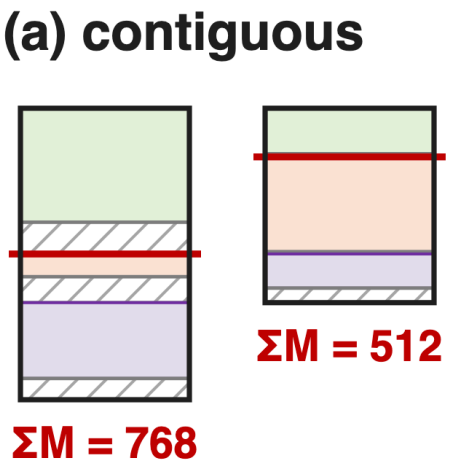
Masked layout: fixed slots, counts on the GPU



m_grouped_fp8_gemm_nt_masked((A, sfa), (B, sfb), D, masked_m, expected_m) · A FP8 [G, max_m, K] · B FP8 [G, N, K] · D BF16 [G, max_m, N]

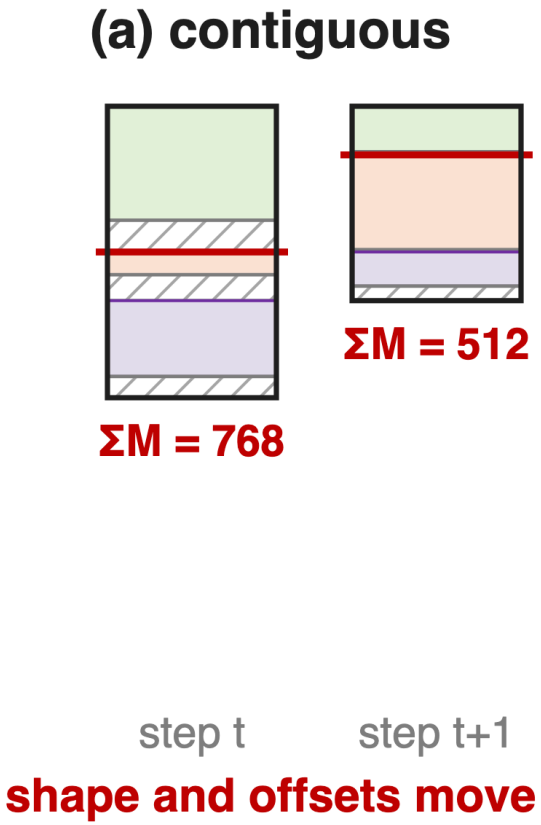
❶ Fixed slots keep every pointer and shape static

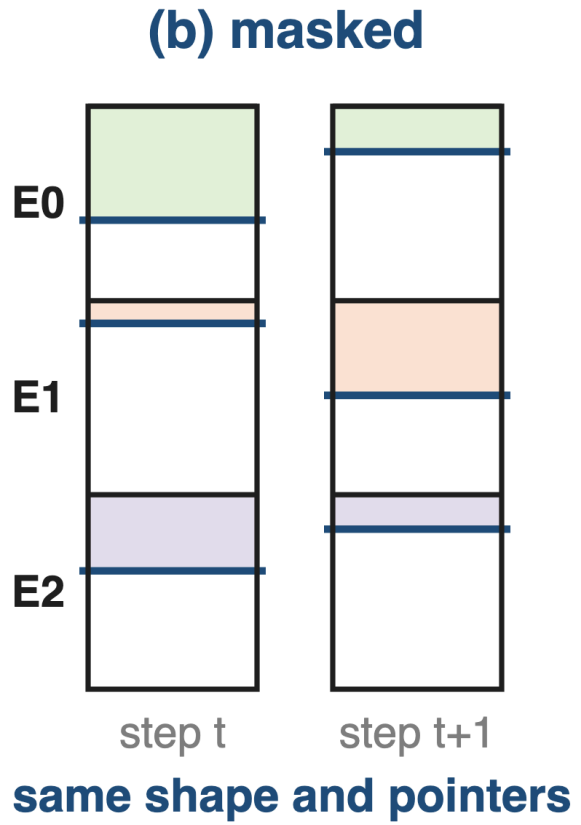
- Contiguous: ΣM and offsets change each step
- A captured kernel would read stale shapes



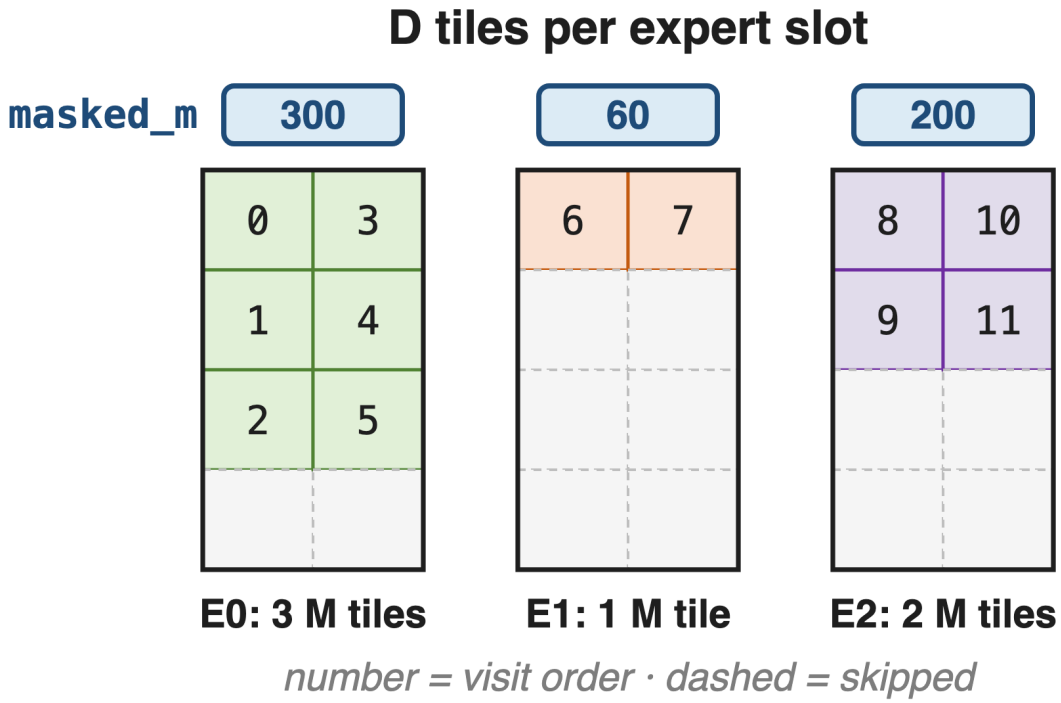
❶ Fixed slots keep every pointer and shape static

- Contiguous: ΣM and offsets change each step
- A captured kernel would read stale shapes
- Masked: slot g starts at **g × max_m**, always
- Only `masked_m` changes between steps





❷ The scheduler reads masked_m on the device



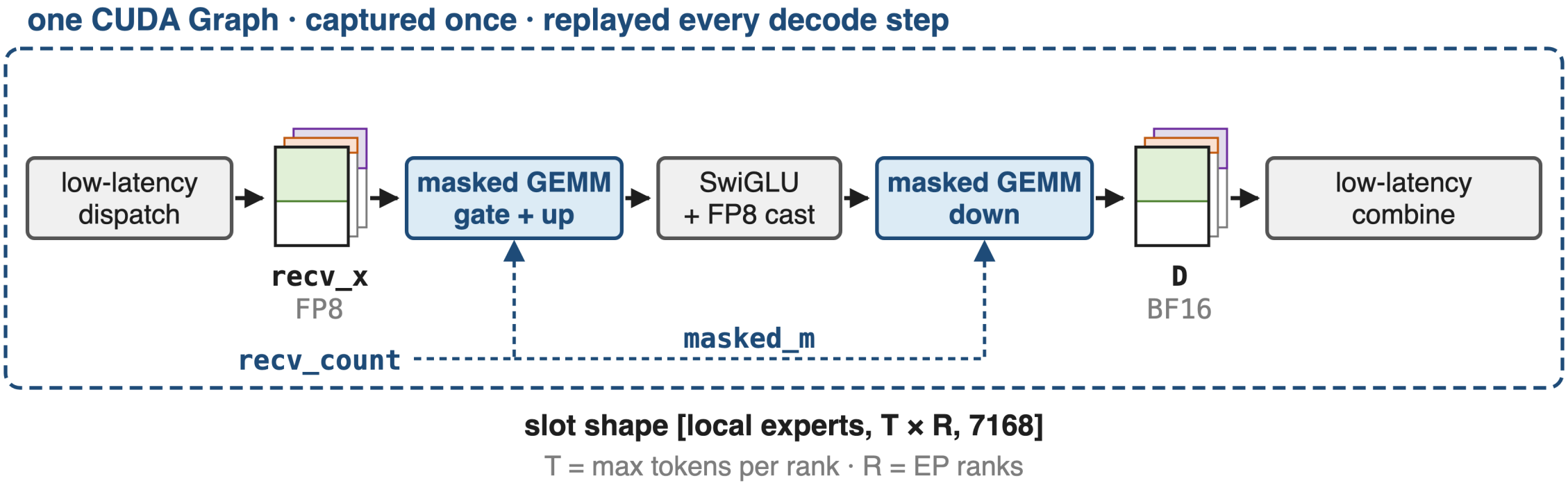
grid = #SMs at every step

```
for t in range(sm, total, SMs):      persistent
  g, m, n = locate(t, masked_m)      GPU read
  if g == G: break                   done
  acc = A[g][m] @ B[g][n].T          FP8 MMA
  D[g][m, n] = bf16(acc)             TMA store
```

locate: group g owns ceil(masked_m[g] / 128) M tiles

The launch never changes; how much work each CTA finds does

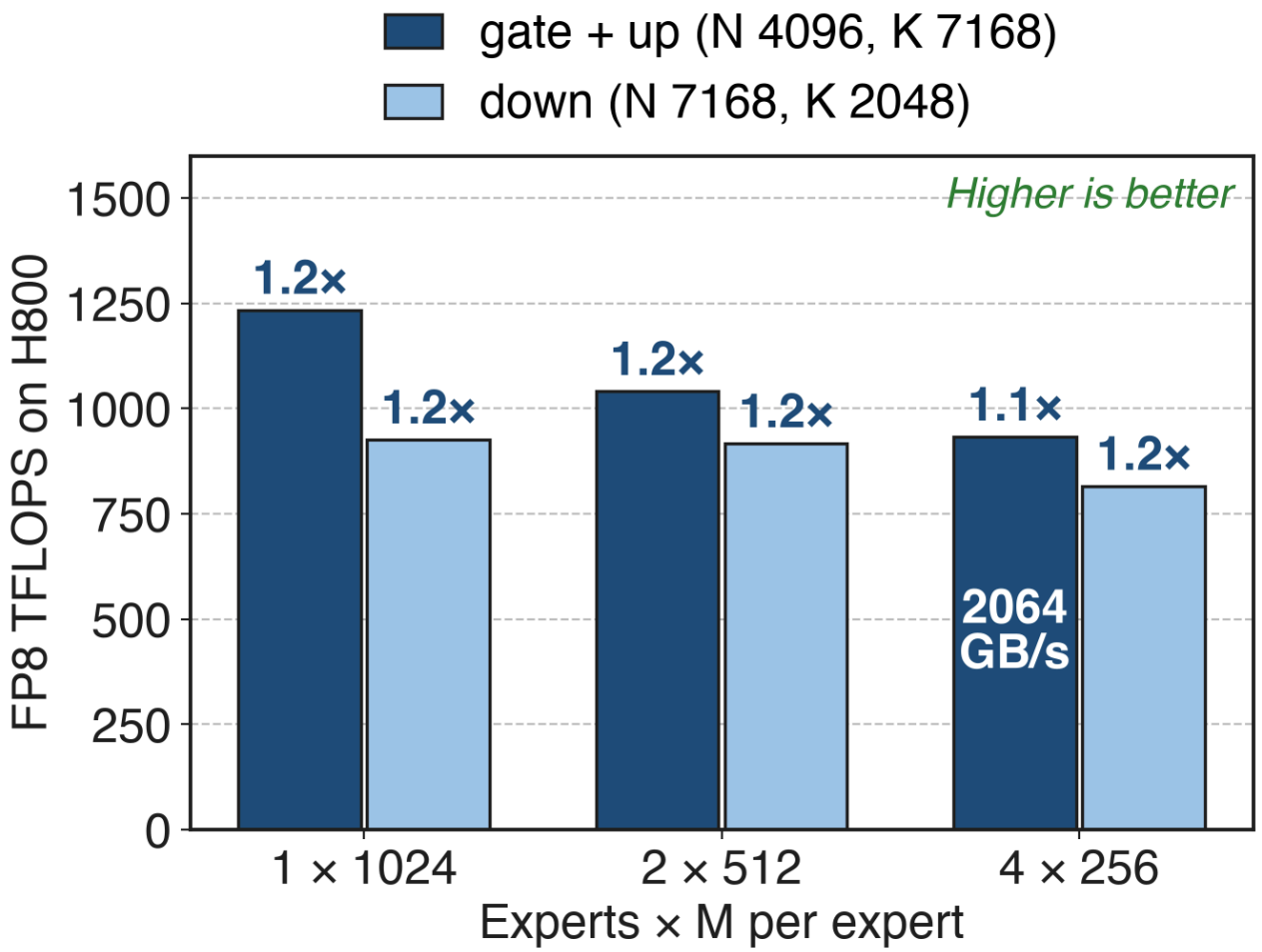
❸ DeepEP low-latency buffers plug straight in



recv_count becomes masked_m; expected_m is the only host input, a hint for the JIT config

Masked GEMM keeps 1.1–1.2× at decode sizes

- **815–1233** TFLOPS on H800
- vs a tuned CUTLASS 3.6 kernel
- At **4 × 256**: weight-streaming, **2064** GB/s



Prefill takes contiguous, decode takes masked

- **Prefill**: normal dispatch
- Contiguous layout (deck 33)
- **Host counts, no CUDA Graph**

- **Decode**: low-latency dispatch
- **Masked layout**
- **Fixed buffers, CUDA Graph**

SGLang runs both via PD disaggregation: 22,282 decode tokens/s per H100 node (EP72, full stack)

Static slots cost memory: 32× the balanced load

- **Slots sized for the worst case**: **32×** the balanced load
- Overflow past capacity is an error
- DeepEP: keep tokens per rank **< 256**
- Low `expected_m`: slower config
- Successor: psum layout in DeepEP V2.5
- SM100 small batches: Mega MoE (deck 35)

Summary

- Host-shaped expert GEMMs: **58** count syncs per decode step
- **Insight**: graphs freeze arguments, not memory: counts stay on the GPU
- Fixed slots + `masked_m` + persistent grid: graph-safe
- **815–1233** TFLOPS on H800, **1.1–1.2×** over CUTLASS

https://github.com/deepseek-ai/DeepGEMM